

BUENAS PRÁCTICAS DE SEGURIDAD AL MOMENTO DE CODIFICAR



	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	2 DE 11

Tabla de contenido

1. OBJETIVO.....	3
2. ALCANCE.....	3
3. DEFINICIONES	3
4. RESPONSABLE.....	3
5. REGISTROS GENERADOS.....	3
6. CONSIDERACIONES GENERALES	3
7. DESCRIPCION.....	4
CONTROL DE CAMBIOS.....	Error! Bookmark not defined.
CUADRO DE APROBACIÓN	Error! Bookmark not defined.

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	3 DE 11

1. OBJETIVO

Suplir de una guía a los Developers para que implementen buenas prácticas de desarrollo durante la etapa de desarrollo.

2. ALCANCE

Con esta guía vamos a evitar que el área de desarrollo cometa errores de seguridad que son bien conocidos en la industria, SQL Injection, Configuración Insegura, Anti Patrones de seguridad, entre otros

3. DEFINICIONES

- Código Fuente: Son los archivos que contienen instrucciones concretas, escritas en algún lenguaje de programación, que compilados, dan como resultado una aplicación que ejecuta lo anteriormente escrito.
- SQL Injection: es una vulnerabilidad que permite al atacante enviar o "inyectar" instrucciones SQL de forma maliciosa y malintencionada dentro del código SQL programado para la manipulación de bases de datos.

4. RESPONSABLE

- Area de Desarrollo

5. REGISTROS GENERADOS

- N/A

6. CONSIDERACIONES GENERALES

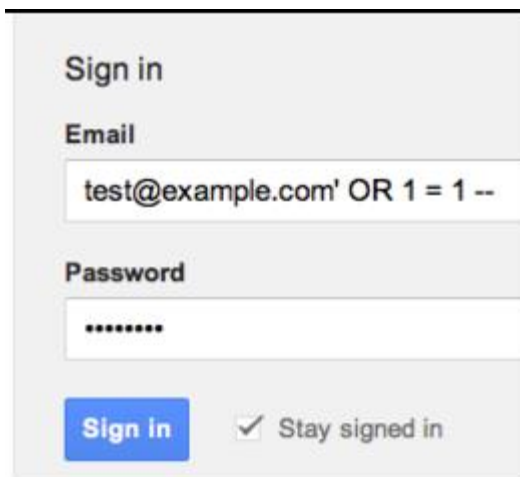
	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	4 DE 11

- Esta guía de seguridad se debe seguir en cada uno de los desarrollos de la plataforma Indigo Vie, Adicional se debe tener en cuenta al momento de hacer Code Review en los Pull Request

7. DESCRIPCIÓN

SQL Injection

Resumiendo, este tipo de ataques este ocurre cuando se introducen a la aplicación caracteres especiales que pueden ser mal interpretados, estos caracteres si no se han tenido en cuenta pueden alterar la consulta SQL que el desarrollador pretende hacer a la base de datos, Por ejemplo.



Sign in

Email

test@example.com' OR 1 = 1 --

Password

Sign in ☒ Stay signed in

Para protegerte de este tipo de ataque, debes erradicar las siguientes malas practicas en tu código:

1. No hagas consultas a la base de datos que concatenen variables de texto (Strings), Usa una librería especializada de acceso a base de datos que construya las consultas por ti, en este tipo de librerías puedes introducir las variables de una manera parametrizada, de esta forma la librería se encarga de manejar los caracteres especiales de manera literal, sin que estos puedan causar daños

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	5 DE 11

Ejemplo Código Vulnerable: La variable <user> se concatena sin ningún control, Si el código anterior a este punto tampoco ha controlado nada, este seria un grave error de seguridad ya que es muy vulnerable a SQL Injection

```

C#
1 public IActionResult Authenticate(string user)
2 {
3     string query = "SELECT * FROM Users WHERE Username = '" + user
4 + "'";
5     // an attacker can bypass authentication by setting user to
6     // this special value
7     // user = "'" or 1=1 or ''='';
8     var userExists = false;
9     if (_context.Database.ExecuteSqlCommand(query) > 0) //
10 Noncompliant
11 {
12     userExists = true;
13 }
14 return Content(userExists ? "success" : "fail");
15 }

```

Ejemplo de código no explotable: la variable <param> aquí es introducida por el método nativo de C# <ExecuteSqlCommand> del objeto Entity.DataBabase, los desarrolladores de esta librería ya se encargan de tratar la codificación de forma correcta para cualquier juego de caracteres que venga en la variable

```

C#
1 public void Foo(DbContext context, string query, string param)
2 {
3     context.Database.ExecuteSqlCommand("SELECT * FROM mytable WHERE
4 mycol=@p0", param); // Compliant, it's a parametrized safe query

```

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	6 DE 11

- Validación de datos de entrada: se trata de controlar cada entrada de datos que tiene la aplicación, el tamaño y forma de estos, para que tenga sentido. Por ejemplo, en el campo teléfono se debe validar que el siguiente valor **no** sea un input valido "exec sp_createuser 12"
- Usa el principio de mínimo privilegio en la configuración del acceso a la base datos, es decir que la aplicación disponga de los mínimos privilegios que necesite, por ejemplo, el usuario de la base datos no debe ser el root de la DB, que por ejemplo el mismo usuario de SQL no tenga acceso a todas las base de datos de nuestros clientes o el usuario de reportes (**xpo**) solo debe tener acceso a la base de datos en modo lectura, etc.
De esta manera si un ataque tiene éxito, comprometerá la mínima cantidad de datos posible.

Configuración insegura:

En el proyecto no deben encontrarse secretos, como contraseñas o keys disponibles en texto plano, ni hardcodeadas en el código, Esta es una mala practica, ya que cualquier usuario con permiso de lectura sobre el código fuente tendrá acceso a dichas llaves. Una de las posibles soluciones para este problema es almacenar los valores secretos en variables de entorno dentro de la maquina y hacer referencia desde el código a estas variables de entorno, con esto evitamos que se conozcan los valores reales de esas llaves.

Ejemplo mala práctica: En este caso se esta exponiendo de forma directa en el web.config datos sensibles sobre servidores y sus credenciales

```
<connectionStrings>
  <!-- Desarrollo Indigo Azure -->
  <add name="CONX_GENESIS" connectionString="{0}data source=ssindigodev.database.windows.net;Initial Catalog={1};User ID=test;
</connectionStrings>
<appSettings>
```

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	7 DE 11

Ejemplo buena práctica: En este segundo caso no se almacenan los datos sensibles directamente en el web.config, en cambio si deja formateado el String para poderlos asignar desde una variable de entorno a través del Código

```
<connectionStrings>
  <!-- Desarrollo Indigo Azure -->
  <add name="CONX_GENESIS" connectionString="{0}data source={1};Initial Catalog={2};User ID={3}; Password={4}; L
</connectionStrings>
```

Name	Value	Source
APPINSIGHTS_INSTRUMENTATIONKEY	 Hidden value. Click to show value	App Service Config
APPLICATIONINSIGHTS_CONNECTION_STRING	 Hidden value. Click to show value	App Service Config
ApplicationInsightsAgent_EXTENSION_VERSION	 Hidden value. Click to show value	App Service Config
password	 testPassword	App Service Config
server	 ssindigodev.database.windows.net	App Service Config
user	 testUser	App Service Config
WEBSITE_LOAD_USER_PROFILE	 Hidden value. Click to show value	App Service Config
WEBSITE_TIME_ZONE	 Hidden value. Click to show value	App Service Config
XDT_MicrosoftApplicationInsights_Mode	 Hidden value. Click to show value	App Service Config

En este caso, las variables de entorno y sus valores quedan guardados en azure, con lo cual se reduce el numero de personan que tienen acceso a esta información (Solo el Admin)

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	8 DE 11

Exposición de información técnica a través de errores

El correcto manejo de errores forma parte de la seguridad básica de la aplicación, El ataque siempre comienza desde la etapa de reconocimiento, durante esa etapa el atacante intenta recopilar la mayor cantidad de información técnica sobre el sistema, como SO, Servidor de servicios, Framework, Librerías, DBs, etc.

Para evitar que este tipo de información sea expuesta cuando haya un error en el sistema, se debe deshabilitar la opción customErrors en el web.config de nuestros servicios.

Error visualizado cuando la opción customErrors está habilitada

Server Error in '/' Application.

Could not load type 'Umbraco.Web.WebApi.UmbracoApiController' from assembly 'umbraco, Version=1.0.4898.17346, Culture=neutral, PublicKeyToken=null'.

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: System.TypeLoadException: Could not load type 'Umbraco.Web.WebApi.UmbracoApiController' from assembly 'umbraco, Version=1.0.4898.17346, Culture=neutral, PublicKeyToken=null'.

Source Error:

An unhandled exception was generated during the execution of the current web request. Information regarding the origin and location of the exception can be identified using the exception stack trace below.

Stack Trace:

```
[TypeLoadException: Could not load type 'Umbraco.Web.WebApi.UmbracoApiController' from assembly 'umbraco, version=1.0.4898.17346, culture=neutral, PublicKeyToken=null'.]
System.Reflection.RuntimeAssembly.GetExportedTypes(RuntimeAssembly assembly, ObjectHandleOnStack retTypes) +0
System.Reflection.RuntimeAssembly.GetExportedTypes() +122
Umbraco.Core.TypeFinder.GetTypeWithFormattedException(Assembly a) +83
Umbraco.Core.TypeFinder.GetClasses(Type assignTypeFrom, IEnumerable<IAssembly> assemblies, Boolean onlyConcreteClasses, Func<2 additionalFilter>) +498
Umbraco.Core.TypeFinder.GetClasses(Type assignTypeFrom, IEnumerable<IAssembly> assemblies, Boolean onlyConcreteClasses, Func<2 additionalFilter>) +1293
Umbraco.BusinessLogic.Actions.Action.<RegisterActionsAndHandlers>b__1() +65
Umbraco.Core.ObjectResolution.LazyManyObjectsResolverBase`2.<get_InstanceTypes>b__5(Func<1 x>) +11
System.Linq.<SelectManyIterator>d__14.MoveNext() +267
System.Linq.Buffer`1..ctor(IEnumerable<1 source>) +488
System.Linq.Enumerable.ToArray(IEnumerable<1 source>) +104
Umbraco.Core.ObjectResolution.LazyManyObjectsResolverBase`2.get_InstanceTypes() +676
Umbraco.Core.ActionsResolver.CreateInstance() +85
Umbraco.Core.ObjectResolution.ManyObjectsResolverBase`2.get_Values() +678
Umbraco.BusinessLogic.Actions.Action.GetJavaScriptFileReferences() +32
Umbraco.Cms.presentation.Umbraco.RenderActionJS() +85
Umbraco.Cms.presentation.Umbraco.Page_Load(Object sender, EventArgs e) +1786
System.Web.Util.CalliHelper.EventArgFunctionCaller(IntPtr fp, Object o, Object t, EventArgs e) +25
Umbraco.BasePages.BasePage.OnLoad(EventArgs e) +19
System.Web.UI.Control.LoadRecursive() +71
System.Web.UI.Page.ProcessRequestMain(Boolean includeStagesBeforeAsyncPoint, Boolean includeStagesAfterAsyncPoint) +3048
```

Version Information: Microsoft .NET Framework Version:4.0.30319; ASP.NET Version:4.0.30319.1

```
1 <?xml version="1.0"?>
2 <configuration>
3     <system.web>
4         <customErrors mode="Off"/>
5     </system.web>
6 </configuration>
```

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	9 DE 11

Anti-patrón: Ignorar errores del código

Ignorar los errores y el comportamiento no esperado de la aplicación es una mala práctica, que, aunque muy obvia puede llegar al entorno de producción por un olvido o falta de revisión del código. En el caso de tener lugar un incidente de seguridad, el log de aplicación será uno de los recursos más básicos y principales para poder dar una adecuada respuesta a este mientras sucede, adicional esta información del log nos ayudará a realizar un correcto análisis post mortem

Ejemplo del problema:

```

1 int[] numbers = new int[2];
2 try
3 {
4     numbers[0] = 23;
5     numbers[1] = 32;
6     numbers[2] = 42;
7
8     foreach(int i in numbers)
9         Console.WriteLine(i);
10 }
11 catch(Exception ex)
12 {
13     // No hacer nada
14 }

```

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	10 DE 11

Ejemplo código seguro:

```

1 var mensaje = "";
2 int[] numbers = new int[2];
3 try
4 {
5     numbers[0] = 23;
6     numbers[1] = 32;
7     numbers[2] = 42;
8
9     foreach(int i in numbers)
10         Log.Debug(i.ToString());
11 }
12 catch(IndexOutOfRangeException ex)
13 {
14     mensaje = "Error message: " + ex.Message;
15     // Siempre a log todas las excepciones
16     Log.Error("Filling the array - An index was out of range!: " +
17         ex.ToString());
18 }
19 catch(Exception ex)
20 {
21     mensaje = "Error message: " + ex.Message;
22     Log.Fatal("Some sort of not expected error occurred: " +
23         ex.ToString());
24     /* En los casos en los que debamos controlar la ocurrencia de un
25     error, pero aún así debamos seguir adelante con el, usaremos throw sin
26     más:
27     - throw ex; MAL relanza la exception pero limpia el stack trace
28     - throw; BIEN relanza la exception manteniendo el stack trace original
29     */
30 }
31 finally
32 {
33     // Si tuvieramos que liberar algún recurso o realizar alguna acción
34     final, esta es el lugar
35 }

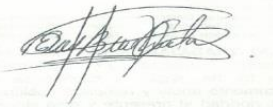
```

	SISTEMA INTEGRADO DE GESTIÓN	CODIGO:	IM-P-020
		VERSIÓN:	01
	Guía Buenas Prácticas de Seguridad	FECHA:	26/07/22
		PAGINA:	11 DE 11

CONTROL DE CAMBIOS DEL DOCUMENTO

Versión No.	Fecha de Aprobación	Descripción de los Cambios
01	26-07-2022	Creación del documento

CUADRO DE APROBACION

ELABORÓ	REVISÓ	APROBÓ
		
Edilma Brand Quintero	José Reyes Páez	José Reyes Páez
Quality	CEO	CEO